Razonamiento Automático 2026/27 Challenge 2: Soluciones de entrenamiento

Programar en ensamblador de Game Boy

> ⚠ **Cómo usar este documento**
> Intenta primero los 6 ejercicios por tu cuenta. Compara después con esta solución **y revísala críticamente**: pueden haber soluciones distintas, pero deben hacer correctamente lo que el ejercicio pide.

> ⚡ **Muy importante**
> Utiliza el emulador y el depurador para comprobar que tu solución hace lo que debe. Es posible hacer códigos muy diferentes, pero debes asegurar 2 cosas: 1) que son correctos, ensamblan y ejecutan y, 2) que el resultado es el que se pide, sin errores, cambios ni añadidos.
>
> Tu mejor herramienta es el depurador del emulador: aprende a usarlo cuanto antes y ***te ahorrará horas de trabajo***, además de hacerte ***aprender muchísimo***.

Ejercicio 1: dos bandas de tile `$0F`

> ⓘ **Cómo se calculan estas direcciones**
> El tilemap empieza en la dirección `$9800` de la VRAM. Los tiles tenemos que ponerlos en las filas 7 y 9 (la primera es la fila 0). Dentro de la fila, el primer tile de los 3 está en la columna 15. Como cada fila ocupa 32 tiles (`$20` es 32 en hexadecimal), el cálculo es:
>
> ```
> DEF LOGO_START = $9800 + FILA*$20 + COLUMNA
> ```

Este cálculo ***es fijo***: el resultado ***nunca varía***, no depende de nada que pueda cambiar. Por eso, el programa ensamblador puede calcularlo, mientras ensambla, y poner el resultado directamente en el código (como si lo calculases tú y lo pusieras a mano). Eso nos permite definirla con una operación matemática ***constante***.

```
    DEF LOGO_START_1 = $9800 + 7*$20 + 15 ;; $98EF
    DEF LOGO_START_2 = $9800 + 9*$20 + 15 ;; $992F
    DEF TILE = 15 ;; $0F

    ;; Top ribbon
    ld  hl, LOGO_START_1  ;; Row 7, Column 15
    ld   a, TILE
    ld  [hl+], a   ;; Tile 1
    ld  [hl+], a   ;; Tile 2
    ld  [hl+], a   ;; Tile 3

    ;; Bottom ribbon
    ld  hl, LOGO_START_2  ;; Row 9, Col 15
    ld  [hl+], a   ;; Tile 1 (A is already $0F)
    ld  [hl+], a   ;; Tile 2
    ld  [hl+], a   ;; Tile 3
```

Ideas clave:

- `ld [nnnn], a` está prohibido a propósito: obliga a usar `hl` como puntero (`ld hl, dirección` + `ld [hl+], a`), que es el patrón que vas a usar todo el curso para tocar VRAM/tilemap — mucho más barato en bytes que la forma absoluta.
- Como las 3 escrituras de cada banda son al mismo tile (`$0F`), no hace falta releer ni recalcular `a` entre una y otra: cárgalo una vez y reutilízalo.
- Si prefieres un bucle en vez de 3 líneas repetidas, también es válido — con solo 3 tiles por banda, ambas formas son razonables; no hay una "más correcta" aquí.

Ejercicio 2: corregir el "Row filler"

Row filler (correcto)

```
def LOGO_LINE_1 equ $9900
def LOGO_LINE_2 equ $9920
def LINE_SIZE   equ 20
def BLANK_TILE  equ 0
;;---------------------------------------
;; Fills a Row with Tiles
;; 📥 Input Values
;;  HL: Start of the row
;;   A: Tile to be written
;;   B: Number of tiles to write
;; ❌ Modifies: HL, B
fill_row:
      ld [hl+], a ;; Clear tile, HL+=1
      dec b       ;; --B (One tile less)
   jr nz, fill_row
   ret            ;; Return

;;---------------------------------------
;; Clear Nintendo Logo
ej2::
   call wait_vblank ;; Synchronize with VBLANK

   ;; Clear Upper Row
   ld  a, BLANK_TILE  ;; A  = Blank tile Number
   ld hl, LOGO_LINE_1 ;; HL = VRAM Start of Upper Line
   ld  b, LINE_SIZE   ;; B = 20 tiles to be cleared
   call fill_row
   ;; Clear Lower Row
   ld  a, BLANK_TILE   ;; A  = Blank tile Number
   ld  hl, LOGO_LINE_2 ;; HL->Nintendo Logo: Lower Row
   ld  b, LINE_SIZE    ;; 20 tiles to be cleared
   call fill_row

   di   ;; Stop program
   halt
```

Lista de errores (9, en la "Versión con errores" del enunciado)

1. `ld hl+, [a]` (debería ser `ld [hl+], a`) — operandos completamente invertidos: `hl+` no es un destino válido de esa forma, y `[a]` trata `A` como si fuera un puntero

de memoria, cosa que no existe en esta CPU.

2. `sub b, 1` (debería ser `dec b`) — `sub` solo admite **un** operando y siempre resta sobre `A`, nunca sobre `B`. Esta línea ni siquiera decrementa el contador que se supone que debe decrementar.
3. `jr z, fill_row` (debería ser `jr nz, fill_row`) — condición invertida: el bucle debe repetir **mientras queden tiles** (`b != 0`), no solo cuando `b` llega a 0.
4. `jr wait_vblank` (debería ser `call wait_vblank`) — con `jr` la ejecución salta dentro de `wait_vblank` pero nunca vuelve a `ej2::` al terminar; rompe la relación llamada/retorno.
5. `ld a, [BLANK_TILE]` (debería ser `ld a, BLANK_TILE`) — `BLANK_TILE` es una constante (`equ 0`), no una dirección de memoria. Queremos introducir el valor `0` en `a` (es decir, lo que vale `BLANK_TILE`). `[BLANK_TILE]` leería la posición `0000` de la memoria y copiaría ese valor leído a `a` (no es lo que queremos).
6. `ld [hl], LOGO_LINE_1` (debería ser `ld hl, LOGO_LINE_1`): intenta escribir el número `LOGO_LINE_1` en la dirección que indica `hl` (que ni siquiera hemos inicializado todavía). Lo que queremos es poner el número `LOGO_LINE_1` en `hl`, para luego acceder a memoria usando `hl`.
7. `ld LINE_SIZE, b` (debería ser `ld b, LINE_SIZE`) —operandos invertidos. La instrucción no tiene sentido: pretende leer `b` y "escribir" en un número (`LINE_SIZE` es el número 20, así que estamos intentado `ld 20, b`, escribir "en" el número 20). Lo que queremos es poner el número `20` en `b`.
8. `jp fill_row` (debería ser `call fill_row`) — igual que el error 4: con `jp` no hay retorno, así que tras vaciar la fila superior el programa nunca continúa con "Clear Lower Row".
9. `ld [LOGO_LINE_2], hl` (debería ser `ld hl, LOGO_LINE_2`) — mismo patrón que el error 6.

> **Por qué importa**
> Casi todos estos errores comparten una causa raíz — **confundir el orden de los operandos o el modo de direccionamiento** (cuándo hace falta `[...]` y cuándo no). Es exactamente lo que entrena el Ejercicio 3: si te cuesta ver estos errores, repásalo primero.

Ejercicio 3

3.1: ¿Qué se lee y qué se escribe?

INSTR.	LEE	ESCRIBE
`ld e, $30`	IMM(`$30`)	Reg(`E`)
`cp %00000010`	Reg(`A`), IMM(`%00000010`)	Reg(`F`):Flags(`Z` , `C`)
`ld [hl], b`	Reg(`B`), Reg(`HL`)	MEM(`HL`)
`jp $0150`	IMM(`$0150`)	Reg(`PC`)
`sub b`	Reg(`A`), Reg(`B`)	Reg(`A`), Reg(`F`):Flags(`Z` , `C`)
`jr z, $08`	Reg(`F`):Flag(`Z`), Reg(`PC`), IMM(`$08`)	Reg(`PC`)**?**
`add [hl]`	Reg(`A`), Reg(`HL`), MEM(`HL`)	Reg(`A`), Reg(`F`):Flags(`Z` , `C`)
`ld a, [bc]`	Reg(`BC`), MEM(`BC`)	Reg(`A`)

3.2: de la lectura/escritura a la instrucción

LEE	ESCRIBE	QUÉ HACE	INSTRUCCIÓN
IMM(`$22`)	Reg(`C`)	Carga el valor inmediato `$22` en `C`	`ld c, $22`
Reg(`A`), IMM(`%00000101`)	Reg(`F`):Flags(`Z` , `C`)	Compara `A` con el valor inmediato `%00000101`	`cp %00000101`
Reg(`E`), Reg(`HL`)	MEM(`HL`)	Guarda el valor de `E` en la dirección que apunta `HL`	`ld [hl], e`
IMM(`$0200`)	Reg(`PC`)	Salta de forma incondicional a `$0200`	`jp $0200`
Reg(`A`), Reg(`D`)	Reg(`A`), Reg(`F`):Flags(`Z` , `C`)	Suma `D` a `A`	`add d`
Reg(`F`):Flag(`Z`), Reg(`PC`), IMM(`$0A`)	Reg(`PC`)**?**	Si `Z` **no** está activo, salta relativo `$0A`	`jr nz, $0A`
Reg(`A`), Reg(`HL`), MEM(`HL`)	Reg(`A`), Reg(`F`):Flags(`Z` , `C`)	Resta a `A` el byte en `[HL]`	`sub [hl]`
Reg(`DE`), MEM(`DE`)	Reg(`A`)	Carga en `A` el byte en `[DE]`	`ld a, [de]`

> **Nota**
> En varias filas de 3.2 hay más de una instrucción igual de correcta. El `?` en `Reg(PC)?` (Ej. 3.2, fila 6) indica que `PC` solo se escribe si la condición del salto se cumple.

Ejercicio 4: leer el PAD sin usar `A`

```
wait_till_B_is_pressed:
   ld  hl, $FF00
   ld  b, $10
   ld  [hl], b     ;; Select Buttons Group 1 (Select, Start, B, A)

   ld  b, [hl]     ;;
   ld  b, [hl]     ;; Read buttons, stabilizing input
   ld  b, [hl]     ;;

   bit 0, b                    ;; Check Bit 0 of B (button A)
jr nz, wait_till_B_is_pressed ;; repeat while A is not pressed
;; A is pressed, act
```

Ideas clave:

- La clave de este ejercicio es una restricción real de la CPU que no está escrita en ningún sitio del material: `ld [nnnn], a` / `ld a, [nnnn]` (direccionamiento absoluto, incluida la forma corta `$FF00`) **solo funcionan con el registro** `A` — no existe una versión con `B`, `C`, etc.
- La salida es usar direccionamiento **indirecto por** `HL`: `ld r, [hl]` y `ld [hl], r` sí aceptan cualquier registro de 8 bits. Cargando `hl = $FF00` una sola vez al principio, se puede leer/escribir ese puerto con `B` en vez de `A` el resto de la rutina.
- `bit 0, b` funciona igual que `bit 0, a`: la instrucción `bit` acepta cualquier registro como operando.

> **⚡ Error típico**
> Intentar sustituir directamente `a` por `b` en `ld [$FF00], a` → `ld [$FF00], b`, sin darse cuenta de que esa forma de la instrucción no existe para `B` — hay que cambiar de estrategia, no solo cambiar el nombre del registro.

Ejercicio 5: logo maligno

> **ⓘ Cómo se lee el volcado de VRAM**
> En la figura, `$9900` (fila 8, columnas 3-15) contiene `19 18 17 16 15 14 13 12 11 10 0F 0E 0D` y `$9920` (fila 9, columnas 4-15) contiene `0C 0B 0A 09 08 07 06 05 04 03 02 01`. Son los mismos índices de tile que el logo normal, **pero en**

orden descendente según la columna crece, en vez de ascendente — de ahí el efecto "maligno": no son tiles distintos, es el mismo logo con el orden invertido.

```
DEF LOGO_FIRST_ROW   = $9903
DEF LOGO_SECOND_ROW_L= $24
DEF LOGO_FIRST_TILE  = $19
DEF LOGO_MEDIUM_TILE = $0D

  ld  hl, LOGO_FIRST_ROW
  ld   a, LOGO_FIRST_TILE
  .top_loop:
    ld  [hl+], a
    dec a
    cp  LOGO_MEDIUM_TILE - 1 ;; stop right after writing MEDIUM_TILE
  jr  nz, .top_loop

  ld  l, LOGO_SECOND_ROW_L   ;; h doesn't change (it's $99):
                             ;; hl = $9924, row 9 column 4
  ;; A is already $0C (LOGO_MEDIUM_TILE - 1): it hasn't been changed
  .bottom_loop:
    ld  [hl+], a
    dec a
  jr  nz, .bottom_loop  ;; Stop when A = 0, not writing the 0
```

Ideas clave:

- No hace falta releer el enunciado como "24 valores sueltos": es un único patrón (contar hacia atrás) aplicado a dos filas consecutivas — mismo bucle, ejecutado dos veces con distinto punto de partida.
- ***Detalle elegante*** (no obligatorio, pero vale la pena entenderlo): el segundo bucle reutiliza el valor que `a` tiene al salir del primero (`$0C`) en vez de recargarlo —al parar el primer bucle exactamente en `$0C` , ese es también el primer valor que toca escribir en la fila de abajo. Ahorra una instrucción, pero solo funciona si lo piensas así de entrada; si prefieres recargar `a` explícitamente con `ld a, $0C` antes del segundo bucle, es igual de correcto y más fácil de razonar bajo presión de tiempo.
- El truco general para cualquier "logo maligno": mira el volcado, identifica si los valores suben, bajan, o siguen otro patrón respecto al logo normal, y tradúcelo a inicialización + condición + paso — igual que cualquier otro bucle de este Challenge.

⚡ Error típico

Usar `jr z` en vez de `jr nz` en cualquiera de los dos bucles (para en el primer tile en vez de recorrer todos). Piensa bien cuándo debe producirse el salto y cuando debe dejar de producirse.

Ejercicio 6: rutina de colisiones

check_collision solution

```
;;-----------------------------------------------
;; Checks if two bytes in memory are equal
;; 📥 Input Values
;; - BC: address of byte 1
;; - DE: address of byte 2
;; ↩️ Return value --
;; - Flag Z if both are equal
is_equal_vBC_vDE:
 ld  a, [bc]
 ld  b, a
 ld  a, [de]
 cp  b
 ret

;;-----------------------------------------------
;; Checks collisions between player and enemy
;;   Collision if (px, py) == (ex, ey)
;;   (x,y) are tile coordinates
;; ↩️ Return value --
;;   A  = 0, when NOT colliding
;;   A != 0, when colliding
check_collision:
 ld bc, enemy_x
 ld de, player_x
 call is_equal_vBC_vDE
 jr nz, .not_colliding

 ld bc, enemy_y
 ld de, player_y
 call is_equal_vBC_vDE
 jr nz, .not_colliding

 ld a, 1
 ret
 .not_colliding:
 xor a
 ret
```

Ideas clave:

- La restricción del enunciado ("toda lectura/escritura en WRAMo debe usar `bc` o `de`") se cumple en `is_equal_vBC_vDE`: los dos únicos accesos a memoria son `ld a, [bc]` y `ld a, [de]`. Guardar el resultado intermedio en `b` (`ld b, a`) no infringe la regla, porque eso es un registro, no un acceso a WRAM.
- No es necesario que la solución implemente una subrutina como `is_equal_vBC_vDE`.
- `is_equal_vBC_vDE` se reutiliza **dos veces** (para `x` y para `y`) con el mismo patrón de llamada + comprobación. Además, es genérica: se puede usar para cualquier par de direcciones.
- El convenio de retorno pedido (`A=0` sin colisión, `A!=0` con colisión) se resuelve con `xor a` (pone `A=0` en una instrucción) y `ld a, 1` — no hace falta ninguna comprobación adicional, el propio flujo del código ya garantiza cuál de las dos ramas se ejecuta.
- ***Detalle***: usar 2 veces `ret` en este caso es más eficiente que saltar hacia un único `ret` con un `jr` o `jp`. `ret` es una instrucción que sólo ocupa 1 byte (`C9`)

> **⚡ Error típico**
> Querer comprobar hacer una comprobación única con los 4 valores (las 2 `x` y las 2 `y`) en vez de 2 comprobaciones (`x` contra `x`, y luego `y` contra `y`). Además de complicar mucho la lógica, no aprovecha una condición práctica: si las `x` no coinciden, no hace falta mirar las `y`, porque ya no puede haber colisión (han de coincidir ambas).